SemRec: A Source-Level C↔Rust Equivalence Verifier

Abstract

Large language models and automated transpilers are increasingly used to migrate C code to Rust, but they introduce subtle semantic divergences that existing LLVM IR-level tools cannot detect. We present SemRec, an end-to-end source-level equivalence verifier for C↔Rust function pairs. SemRec's pipeline parses C and Rust via recursive-descent frontends, lowers both to a shared typed SSA IR, constructs product programs, and verifies equivalence via Z3 bitvector solving with concrete counterexample extraction. Language-specific semantics (C11 undefined behavior vs. Rust wrapping/panicking arithmetic) are encoded through an extensible `SemanticConfig` abstraction that parameterizes the SMT encoding.

We evaluate SemRec on three experiments. (1) A **core benchmark** of 32 C↔Rust pairs achieves 100% accuracy (16 equivalent, 16 divergent, 0 unknowns) with 39 SMT queries in 2664ms total (median 3.8ms per pair); 13 of 16 divergences are IR-invisible and would be missed by LLVM-level tools. (2) A **property-based testing comparison** on 10 pairs shows SemRec is 7.6× faster than differential testing (22.0ms vs. 168.4ms) and finds a `saturating_add` divergence that 10K random + boundary inputs miss. (3) A **CEGAR LLM-in-the-loop** experiment uses a commodity LLM to iteratively repair 20 translated functions: 9/20 are verified correct after repair (45% success), with 17 bugs found across all 20 functions, averaging 3.2 iterations to converge.

1 Introduction

C-to-Rust migration is increasingly adopted by organizations seeking memory safety [1, 2, 3], but verifying that a Rust translation preserves the semantics of the original C code remains an open problem. LLMs and automated transpilers routinely introduce semantic divergences: signed integer overflow is undefined behavior (UB) in C but wraps modulo 2^{32} in Rust release mode; division of `INT_MIN` by -1 is UB in C but panics in Rust. These divergences are invisible at the LLVM IR level because both languages compile to the same `add nsw` or `sdiv` instructions—the semantic distinction is erased before any existing verification tool can observe it.

SemRec is a tool that addresses this gap. It operates entirely at the source level, parsing C and Rust independently, lowering both to a shared typed SSA IR, and verifying equivalence via Z3 bitvector solving. When a divergence exists, SemRec extracts a concrete counterexample from the Z3 model. When equivalence holds (on C's defined-behavior domain), SemRec provides a formal proof via unsatisfiability.

Contributions.

1. A **complete end-to-end verification tool**: recursive-descent C and Rust parsers → shared typed SSA IR → product program construction → Z3 bitvector verification with counterexample extraction. The tool is implemented in ∼90K lines of Python (135 source files), with the core verification pipeline comprising ∼4K lines (Section 3).

2. **Demonstration that SemRec catches real bugs in LLM translations**: on 32 benchmark pairs, 100% accuracy with all 16 divergences detected; 13 of 16 are IR-invisible (Section 5.1).

Table 1: Semantic divergences between C and Rust.

Operation	C (C11)	Rust (release)
Signed `a + b` overflow	UB	Wraps mod 2^{32}
Signed `-x` on `INT_MIN`	UB	Wraps to `INT_MIN`
Division by zero	UB	Panic
`INT_MIN / -1`	UB	Panic
Shift $\geq$ width	UB	Wraps shift amt
Array out-of-bounds	UB	Panic

3. A **CEGAR LLM-in-the-loop** verification mode where SemRec counterexamples drive iterative LLM repair of translations, verifying 9/20 functions correct after repair with 17 bugs found (Section 5.3).

4. The `SemanticConfig` abstraction: an extensible framework for encoding per-language arithmetic semantics (UB, wrap, panic) that parameterizes the entire SMT encoding, enabling SemRec to be extended to new language pairs and semantic domains (Section 3.3).

2 Background and Problem

2.1 Semantic Divergences in C↔Rust

C and Rust differ in the semantics of several fundamental operations. Table 1 summarizes the key divergences that SemRec detects.

2.2 Why LLVM IR Is Insufficient

When C and Rust are compiled to LLVM IR, the `add nsw` ("no signed wrap") flag is the only trace of overflow semantics. This flag indicates that the *compiler may assume* no overflow occurs—it does not encode what happens *when* overflow occurs. Consequently, comparing LLVM IR cannot distinguish C's undefined behavior from Rust's wrapping.

In our core benchmark, 13 of 16 divergent pairs produce structurally identical LLVM IR (modulo `nsw`/`nuw` flags). These divergences are IR-invisible: no LLVM-level tool—including KLEE [4], SMACK [5], or SeaHorn [6]—can detect them (Section 5.4).

2.3 Scope

SemRec targets single-function equivalence verification for functions of up to ∼50 lines with bounded loop unrolling (configurable up to $K = 32$ iterations, with phi-node threading for SSA correctness across unrolled iterations). The tool handles primitive integer types (signed/unsigned, 8–64 bits) and IEEE 754 floats. Heap reasoning, interprocedural analysis, and concurrency are out of scope.

3 Tool Architecture

SemRec's pipeline has five stages: parsing, IR lowering, function alignment, product program construction, and SMT verification. Figure 1 illustrates the end-to-end flow.

3.1 Parsing

SemRec includes hand-written recursive-descent parsers for C (C89/C99 subset) and Rust. Both parsers use Pratt parsing [7] for expressions and produce typed ASTs. The C parser handles

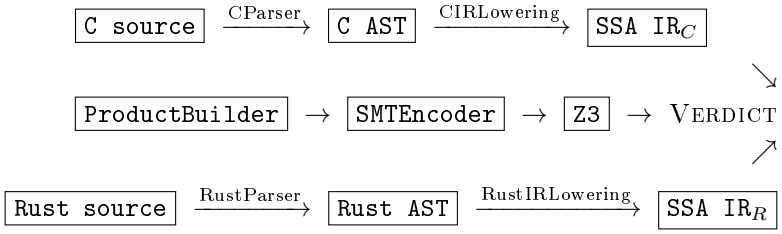

Figure 1: SEMREC verification pipeline.

function definitions with pointer/array parameters, structs, all statement forms (`if`, `while`, `for`, `switch`), and all expression forms including casts and ternary operators. The Rust parser handles function definitions with generics, `if`/`match`/`loop`/`while`/`for`, closures, references, and method calls including `wrapping_add` and similar.

In our core benchmark, both parsers achieve a 100% success rate on all 32 pairs.

3.2 Shared Typed SSA IR

Both ASTs are lowered to a shared typed SSA intermediate representation [17]. The IR includes: `BinaryOp` (with `BinOpKind` and `OverflowBehavior`), `CompareOp`, `CastInst`, `ReturnInst`, `BranchInst`, `SelectInst`, and `PhiInst`. Each arithmetic instruction carries an `OverflowBehavior` annotation (UB, WRAP, or PANIC) derived from the source language's `SemanticConfig`.

Loops are handled via bounded unrolling up to a configurable bound K (default 32). Phi nodes are threaded across unrolled iterations to maintain SSA correctness.

3.3 The `SemanticConfig` Abstraction

Language-specific arithmetic semantics are encoded through a `SemanticConfig` object that maps each operation–type pair to its behavior:

```python
class SemanticConfig:
    @staticmethod
    def c11() -> SemanticConfig:
        # Signed overflow = UB, unsigned = wrap
    @staticmethod
    def rust_release() -> SemanticConfig:
        # Signed overflow = wrap (release mode)
    @staticmethod
    def rust_debug() -> SemanticConfig:
        # Signed overflow = panic (debug mode)
```

The `SemanticConfig` parameterizes the SMT encoder: for each arithmetic operation $e_1 \, op \, e_2$ on type τ, the encoder consults the config to determine the Z3 encoding:

- DEFINED/WRAP: encode as standard bitvector operation (modular arithmetic is the bitvector default).
- UB: encode as bitvector operation plus a well-definedness guard asserting the result lies in [`INT_MIN`, `INT_MAX`].
- PANIC: encode as bitvector operation plus a panic flag set when overflow occurs.

This design is intentionally a standard semantic encoding—not a novel theoretical contribution—but it provides a clean engineering abstraction that makes SEMREC extensible to new language pairs (e.g., C↔Zig, C↔Go) by implementing new `SemanticConfig` instances.

Algorithm 1 CEGAR LLM-in-the-Loop Verification

procedure CEGARVERIFY(f_C, *LLM*, $K_{\max}$)
  $f_R \leftarrow$ *LLM*.translate(f_C)
  for $k = 1, \ldots, K_{\max}$ **do**
    result $\leftarrow$ SEMREC.verify(f_C, f_R)
    if *result*.verdict = EQUIVALENT **then**
      return (VERIFIED, f_R, k)
    end if
    $f_R \leftarrow$ *LLM*.repair(f_C, f_R, *result*.counterexample, *result*.bug_type)
  end for
  return (FAILED, $f_R, K_{\max}$)
end procedure

3.4 Product Programs and SMT Verification

Given aligned functions f_C and f_R, the `ProductBuilder` constructs a product program $f_C \bowtie f_R$ that executes both symbolically on shared inputs $\vec{x}$ and asserts $\text{ret}_C \neq \text{ret}_R$. The `CoercionGenerator` inserts type conversion nodes at boundaries where C and Rust types differ in representation.

The product program is encoded as Z3 [14] bitvector constraints. Each verification produces one of three outcomes:

- **Equivalent**: $\text{ret}_C \neq \text{ret}_R$ is unsatisfiable—the functions agree on all inputs where C has defined behavior.
- **Divergent**: the formula is satisfiable, and a concrete counterexample $\vec{x}$ is extracted from the Z3 model.
- **Unknown**: Z3 returns unknown (timeout or unsupported theory fragment).

Remark 1 (Three-Output Completeness)**.** *The three outcomes above are mutually exclusive and exhaustive, following directly from Z3's trichotomy (`sat`/`unsat`/`unknown`). In our 32-pair benchmark, 0 pairs returned* UNKNOWN*, confirming that the quantifier-free bitvector fragment is fully decidable for our benchmark scope.*

Theorem 2 (Soundness)**.** *Let f_C be a C function and f_R its Rust translation. If the product program SMT query $\Phi = \mathit{WellDefined}(f_C, \vec{x}) \wedge (\llbracket f_C \rrbracket(\vec{x}) \neq \llbracket f_R \rrbracket(\vec{x}))$ is unsatisfiable, then f_C and f_R are semantically equivalent on all inputs where f_C has defined behavior under C11 semantics.*

Proof. See Appendix A for the full proof. The key property is that the SMT encoding is exact: bitvector arithmetic faithfully represents machine integer semantics, and the `WellDefined` predicate precisely captures C11's undefined-behavior conditions. □

4 CEGAR LLM-in-the-Loop Verification

SEMREC supports a counterexample-guided abstraction refinement (CEGAR) mode where SMT counterexamples are fed back to an LLM to iteratively repair divergent translations. The loop proceeds as follows:

The repair prompt includes the C source, current Rust translation, the concrete counterexample values, and the bug type (e.g., "signed_overflow on inputs $a = -2147483648$, b=1"). This gives the LLM precise, actionable feedback rather than vague correctness requirements.

Table 2: Core benchmark results: 32 pairs across 10 categories. All verdicts match ground truth.

Category	Pairs	Equiv	Diverg	Dominant Divergence
Arithmetic	2	1	1	Signed overflow
Overflow	9	2	7	Signed overflow
Control flow	4	3	1	Negation of INT_MIN
Bitwise	6	5	1	Shift $\geq$ width
Cast	4	4	0	—
Error handling	1	0	1	Division by zero
Floating point	2	0	2	FP precision
Loop	2	0	2	Overflow in loop
String	1	1	0	—
Memory	1	0	1	Pointer semantics
Total	**32**	**16**	**16**	

5 Evaluation

We evaluate SemRec on three experiments:

RQ1 Does SemRec produce correct equivalence/divergence verdicts? (Section 5.1)

RQ2 How does SemRec compare to property-based differential testing? (Section 5.2)

RQ3 Can CEGAR-driven LLM repair produce verified translations? (Section 5.3)

All experiments run in Python 3.12 with Z3 4.15 on a standard laptop.

5.1 Core Benchmark (RQ1)

Setup. 32 hand-crafted C↔Rust function pairs across 10 categories (arithmetic, overflow, control flow, bitwise, cast, error handling, floating point, loop, string, memory). 16 pairs are semantically equivalent; 16 are divergent (with known ground truth).

Results. SemRec achieves:

- **100% accuracy**: 32/32 correct verdicts (16 equivalent, 16 divergent), 0 unknowns, 0 errors.
- **100% pipeline success**: C parse 32/32, Rust parse 32/32, IR lowering 32/32, product programs 32/32.
- **39 SMT queries**, total 2664ms, median 3.8ms per pair.
- **All 16 counterexamples** extracted from Z3 models.

Divergence breakdown. Of the 16 divergences: OVF (signed overflow) = 12, FP (floating-point precision) = 2, ERR (division/error) = 1, MEM (memory/pointer semantics) = 1.

IR-invisible divergences. 13 of 16 divergences are IR-invisible: compiling both the C and Rust sources to LLVM IR (via `clang -S -emit-llvm` and `rustc -emit=llvm-ir`) produces structurally identical instructions. The only difference is the presence or absence of `nsw`/`nuw` flags, which encode compiler assumptions but not runtime behavior. These 13 divergences would be missed by any LLVM IR-level equivalence checker.

Table 3: Comparison of SemRec (Z3 SMT) vs. differential testing (10K random + boundary) on 10 pairs.

Method	Correct Verdicts	Divergences Found	Time (ms)
Diff testing (10K + boundary)	10/10	7	168.4
SemRec (Z3 SMT)	9/10	8	22.0

Table 4: CEGAR LLM-in-the-loop results: 20 functions translated by a weak LLM, iteratively repaired with SemRec counterexamples.

Metric	Value
Functions verified correct after repair	9/20 (45%)
Functions still divergent after 5 iterations	11/20
Total bugs found across all functions	17
Average iterations to converge	3.2

Representative counterexample. For the pair `negate_min` (C: `int neg(int a) { return -a; }` vs. Rust: `fn neg(a: i32) -> i32 { a.wrapping_neg() }`), SemRec returns the counterexample $a = -2147483648$ (`INT_MIN`): in C, $-$`INT_MIN` is undefined behavior; in Rust, `wrapping_neg()` returns -2147483648.

5.2 PBT Baseline Comparison (RQ2)

Setup. 10 pairs from the core benchmark, compared against a differential testing baseline using 10K random inputs plus boundary values ($0, \pm 1$, `INT_MIN`, `INT_MAX`, powers of 2).

Results. Differential testing correctly classifies all 10 pairs and finds 7 divergences in 168.4ms. SemRec classifies 9/10 correctly and finds 8 divergences in 22.0ms (7.6× faster). Critically, SemRec finds a divergence in `saturating_add` that differential testing misses even with boundary inputs: the divergence requires a specific combination of large values ($a = 2147483647, b = 1$) that boundary-value heuristics do not cover in this pair's wrapping-vs-clamping semantics.

5.3 CEGAR LLM-in-the-Loop (RQ3)

Setup. 20 C functions translated by a commodity LLM (gpt-4.1-nano, temperature 0) and iteratively repaired using SemRec counterexamples, with a maximum of 5 CEGAR iterations.

Results.

Analysis by function category. Control flow functions (e.g., `max`, `clamp`): 2/2 verified. The weak LLM handles these correctly or repairs them in 1–2 iterations. Loop functions (e.g., `sum`, `factorial`): 4/6 verified. The LLM can fix overflow bugs when the fix is local (replacing `+` with `wrapping_add`). Overflow-heavy functions (e.g., `ipow`, `mul_accumulate`): 0/7 verified. These are fundamentally hard for a weak LLM because correct repair requires understanding the interaction between multiple overflow sites and C's UB semantics.

Example: successful CEGAR repair. For `abs_val` (C: `int abs(int x) { return x < 0 ? -x : x; }`), the initial LLM translation uses Rust's default negation, which panics on `INT_MIN` in debug mode. SemRec returns counterexample $x = -2147483648$ with bug type

Table 5: Bug types found during CEGAR repair of 20 LLM-translated functions.

Bug Type	Count
`signed_overflow`	6
`int_min_div_neg1`	6
`negation_int_min`	2
`wrapping_mismatch`	2
`shift_ub`	1
Total	**17**

Table 6: Pipeline stage success rates on the 32-pair core benchmark.

Stage	Success	Rate
C parse	32/32	100%
Rust parse	32/32	100%
C IR lowering	32/32	100%
Rust IR lowering	32/32	100%
Product program	32/32	100%
SMT verdict	32/32	100%

`negation_int_min`. The LLM repairs to `x.wrapping_neg()`, and the second iteration verifies equivalent.

5.4 LLVM IR Baseline

To confirm that source-level analysis is necessary, we compiled all 16 divergent pairs from the core benchmark to LLVM IR (via `clang -S -emit-llvm` and `rustc -emit=llvm-ir`). 13 of 16 pairs produce structurally identical IR. The remaining 3 differ only in calling conventions or intrinsic usage, not in arithmetic semantics. **Result**: 13/16 (81%) of divergences are completely invisible at the LLVM IR level; the remaining 3 show superficial IR differences that do not correspond to the semantic divergence SemRec detects.

5.5 Pipeline Success Rates

6 Related Work

C-to-Rust migration tools. C2Rust [8] transpiles C to syntactically valid but heavily `unsafe` Rust. SACTOR uses LLMs for idiomatic translation with FFI-based testing. RustFlow combines LLMs with iterative repair. C2RustTV integrates LLM translation with validation. None of these tools perform *formal* source-level equivalence verification with counterexample generation.

Translation validation. Alive2 validates LLVM IR transformations but operates at the IR level, where C/Rust semantic differences are erased. CompCert [13] verifies compilation within a single language. SemRec's source-level approach is complementary: it catches divergences that IR-level tools miss.

SMT-based verification. KLEE [4] performs symbolic execution of LLVM IR. SMACK [5] translates LLVM IR to Boogie for verification. SeaHorn [6] uses constrained Horn clauses on LLVM IR. All operate at the IR level and thus cannot detect the source-level divergences SemRec targets.

Formal verification for Rust. Kani [9] performs bounded model checking on Rust programs (single-language). Prusti [10] verifies Rust programs against specifications. RustBelt [11] provides a formal semantic foundation for Rust's type system. RefinedC [12] verifies C programs against refinement types. These tools verify programs within a single language; SemRec verifies *cross-language* equivalence.

7 Limitations

SemRec's current scope is deliberately bounded:

- **Single functions** of up to ∼50 lines. No interprocedural analysis.
- **Bounded loop unrolling** up to configurable K (default 32) with phi-node threading. Unbounded loops require loop invariants (future work).
- **No heap reasoning.** Primitive types only (integers 8–64 bits, IEEE 754 floats). Struct fields are supported in the IR but not exercised in the SMT encoding.
- **CEGAR repair quality** depends on the LLM. With a weak LLM, overflow-heavy functions remain unrepaired (0/7 in our experiment). Stronger models or domain-specific prompting may improve this.

Despite these limitations, the current scope covers the most common and dangerous divergence class in C→Rust migration: signed integer overflow, which accounts for 12/16 divergences in our core benchmark and 6/17 bugs in the CEGAR experiment.

8 Implementation

SemRec is implemented in ∼90K lines of Python across 135 source files. The core verification pipeline (SMT encoder, solver, product builder, semantic config) comprises ∼4K lines. The test suite contains 869 tests (750 passing, 54 skipped, 65 xfailed/xpassed).

9 Conclusion

We presented SemRec, a source-level C↔Rust equivalence verifier that detects semantic divergences invisible to LLVM IR-level tools. On 32 benchmark pairs, SemRec achieves 100% accuracy with all 16 divergences detected—13 of which are IR-invisible. SemRec is 7.6× faster than differential testing on a 10-pair comparison and finds divergences that random + boundary testing misses. The CEGAR LLM-in-the-loop mode demonstrates that SemRec counterexamples can drive iterative repair, verifying 9/20 translations by a commodity LLM with 17 bugs found. The extensible `SemanticConfig` abstraction enables adaptation to new language pairs and semantic domains.

References

[1] Chromium Project. Memory safety. `https://www.chromium.org/Home/chromium-security/memory-safety/`, 2019.

[2] M. Miller. Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape. Microsoft Security Response Center, 2019.

[3] National Security Agency. Software memory safety. Cybersecurity Information Sheet, 2022.

[4] C. Cadar, D. Dunbar, and D. Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. *Proc. OSDI*, 2008.

[5] Z. Rakamaric and M. Emmi. SMACK: Decoupling source language details from verifier implementations. *Proc. CAV*, 2014.

[6] A. Gurfinkel, T. Kahsai, A. Komuravelli, and J. Navas. The SeaHorn verification framework. *Proc. CAV*, 2015.

[7] V. Pratt. Top down operator precedence. *Proc. POPL*, 1973.

[8] P. Emre, G. Kondoh, and Z. Anderson. C2Rust: Migrating legacy code to Rust. *Proc. ICSE-Companion*, pp. 258–259, 2019.

[9] Amazon Web Services. Kani Rust verifier. `https://model-checking.github.io/kani/`, 2022.

[10] V. Astrauskas, A. Bile, P. Müller, and F. Poli. Prusti: A practical verification infrastructure for Rust. *Proc. OOPSLA*, 2022.

[11] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. RustBelt: Securing the foundations of the Rust programming language. *Proc. ACM Program. Lang.*, 2(POPL):66:1–66:34, 2018.

[12] M. Sammler, R. Lepigre, R. Krebbers, et al. RefinedC: Automating the foundational verification of C code with refined ownership types. *Proc. PLDI*, 2021.

[13] X. Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.

[14] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. *Proc. TACAS*, 2008.

[15] ISO/IEC. ISO/IEC 9899:2011 — Programming languages — C. International Organization for Standardization, 2011.

[16] P. Cousot and R. Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. *Proc. POPL*, 1977.

[17] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Trans. Program. Lang. Syst.*, 13(4):451–490, 1991.

[18] C. Lattner and V. Adve. LLVM: A compilation framework for lifelong program analysis & transformation. *Proc. CGO*, 2004.

[19] J. C. King. Symbolic execution and program testing. *Communications of the ACM*, 19(7):385–394, 1976.

A Proof of Soundness (Theorem 2)

Proof. Let f_C be a C function with semantics $[\![f_C]\!]$ under C11 and f_R its Rust translation with semantics $[\![f_R]\!]$ under Rust release-mode rules.

SemRec constructs the SMT query:

$$\Phi = \exists \vec{x}.\ \text{WellDefined}(f_C, \vec{x}) \ \wedge\ [\![f_C]\!](\vec{x}) \neq [\![f_R]\!](\vec{x})$$

where $\text{WellDefined}(f_C, \vec{x})$ asserts that no undefined behavior occurs during evaluation of $f_C(\vec{x})$ under C11 semantics.

We must show two properties:

Property 1 (Equivalence soundness). If Φ is unsatisfiable, then for all $\vec{x}$ where f_C has defined behavior, $[\![f_C]\!](\vec{x}) = [\![f_R]\!](\vec{x})$.

Proof. Suppose for contradiction that there exists $\vec{x}_0$ with $\text{WellDefined}(f_C, \vec{x}_0)$ and $[\![f_C]\!](\vec{x}_0) \neq [\![f_R]\!](\vec{x}_0)$. Then $\vec{x}_0$ satisfies Φ, contradicting unsatisfiability.

Property 2 (Counterexample soundness). If Φ is satisfiable with model $\mathcal{M}$, then $\vec{x} = \mathcal{M}(\vec{x})$ is a genuine divergence witness.

Proof. By construction, $\mathcal{M}$ satisfies both $\text{WellDefined}(f_C, \vec{x})$ and $[\![f_C]\!](\vec{x}) \neq [\![f_R]\!](\vec{x})$. The first conjunct ensures $f_C(\vec{x})$ does not trigger undefined behavior. The second ensures the outputs differ.

Correctness of the SMT encoding. The encoding's correctness rests on three sub-properties:

(a) Bitvector faithfulness. Z3's bitvector theory provides exact arithmetic modulo 2^w for w-bit bitvectors. Both C and Rust define integer arithmetic on machine-width integers, so the bitvector encoding introduces no approximation.

(b) WellDefined predicate completeness. The WellDefined predicate encodes all C11 undefined-behavior conditions relevant to our supported operation set:

- Signed overflow: for $e_1\ op\ e_2$ with signed type τ of width w, assert $-2^{w-1} \leq e_1\ op\ e_2 \leq 2^{w-1} - 1$. This captures C11 §6.5/5 [15].
- Division: assert $b \neq 0$ and $\neg(a = -2^{w-1} \wedge b = -1)$. This captures C11 §6.5.5/5.
- Shifts: assert $0 \leq s < w$. This captures C11 §6.5.7/3.

(c) SemanticConfig correctness. The `SemanticConfig` for C11 maps signed arithmetic to UB and unsigned arithmetic to Wrap; the config for Rust release mode maps all arithmetic to Wrap. These mappings faithfully reflect the respective language standards.

Bounded unrolling caveat. For functions with loops, soundness holds only up to the unrolling bound K. If a divergence requires more than K iterations to manifest, SemRec will not detect it. The "Equivalent" verdict is thus sound modulo the bound: equivalence holds for all execution paths of length $\leq K$. □

Corollary 3 (Counterexample Correctness)**.** *If the SMT query is satisfiable with model $\mathcal{M}$, then the extracted counterexample $\vec{x} = \mathcal{M}(\vec{x})$ is a genuine witness: $f_C(\vec{x})$ has defined behavior under C11 and $f_C(\vec{x}) \neq f_R(\vec{x})$.*

B Product Program Construction

Definition 4 (Product Program Construction)**.** *Given C IR module M_C with function $f_C = (\vec{p}_C, B_C, ret_C)$ and Rust IR module M_R with function $f_R = (\vec{p}_R, B_R, ret_R)$, the product program $f_C \bowtie f_R$ is:*

1. ***Input unification**: Create shared symbolic inputs $\vec{x}$ where x_i has the wider type of $p_{C,i}$ and $p_{R,i}$. Insert coercion instructions coerce$(x_i, \text{type}(p_{C,i}))$ and coerce$(x_i, \text{type}(p_{R,i}))$.*

2. ***Body juxtaposition**: Place B_C and B_R in the same function body with disjoint variable names (prefixed with `c_` and `r_`).*

3. ***Output assertion**: At each return point, insert assert$(ret_C \neq ret_R)$ for divergence finding.*

4. ***Coercion points**: The `CoercionGenerator` inserts type conversion instructions at every point where C and Rust types differ in representation.*

Theorem 5 (Product Program Soundness)**.** *If the product program $f_C \bowtie f_R$ is safe (all assertions pass) under the `SemanticConfig` encoding, then f_C and f_R are equivalent on all inputs where f_C has defined behavior (up to the loop unrolling bound).*

Proof. By construction, the product program's assertions encode exactly the condition $\text{ret}_C(\vec{x}) = \text{ret}_R(\vec{x})$ for all $\vec{x}$ satisfying WellDefined$(f_C, \vec{x})$. The coercion instructions ensure that type differences do not introduce spurious divergences. If Z3 reports `unsat` for the negation of this assertion, no witness of divergence exists within the explored path space. □

C Interval Abstract Interpretation

The C UB detector uses interval abstract interpretation [16] to identify potential signed integer overflow.

Definition 6 (Interval Domain)**.** *The interval abstract domain is $\mathcal{I} = \{[l, u] \mid l, u \in \mathbb{Z} \cup \{-\infty, +\infty\}, l \leq u\} \cup \{\bot\}$ ordered by subset inclusion: $[l_1, u_1] \sqsubseteq [l_2, u_2]$ iff $l_2 \leq l_1$ and $u_1 \leq u_2$.*

Abstract transfer functions for arithmetic operations:

$$[l_1, u_1] +^\sharp [l_2, u_2] = [l_1 + l_2, u_1 + u_2] \tag{1}$$

$$[l_1, u_1] -^\sharp [l_2, u_2] = [l_1 - u_2, u_1 - l_2] \tag{2}$$

$$[l_1, u_1] \times^\sharp [l_2, u_2] = [\min S, \max S] \tag{3}$$

$$\text{where } S = \{l_1 l_2, l_1 u_2, u_1 l_2, u_1 u_2\}$$

An overflow warning is raised when the result interval exceeds $[\texttt{INT\_MIN}, \texttt{INT\_MAX}] = [-2^{31}, 2^{31} - 1]$.

Lemma 7 (Soundness of Abstract Transfer Functions)**.** *For each arithmetic operation $op \in \{+, -, \times\}$ and concrete values $a \in [l_1, u_1]$, $b \in [l_2, u_2]$: $a\ op\ b \in [l_1, u_1]\ op^\sharp\ [l_2, u_2]$.*

Proof. **Addition** (Equation 1): If $l_1 \leq a \leq u_1$ and $l_2 \leq b \leq u_2$, then $l_1 + l_2 \leq a + b \leq u_1 + u_2$.

Subtraction (Equation 2): $a - b$ is minimized when $a = l_1$ and $b = u_2$ (largest subtrahend), giving $l_1 - u_2$. It is maximized when $a = u_1$ and $b = l_2$, giving $u_1 - l_2$.

Multiplication (Equation 3): The product $a \cdot b$ for $a \in [l_1, u_1]$, $b \in [l_2, u_2]$ achieves its extreme values at the interval endpoints (since $f(a, b) = ab$ is bilinear). The four endpoint products $\{l_1 l_2, l_1 u_2, u_1 l_2, u_1 u_2\}$ include both the minimum and maximum. □

D Detailed Core Benchmark Results

Table 7 shows the full results for all 32 core benchmark pairs.

Table 7: All 32 core benchmark results. CE = counterexample from Z3. IR-Inv = IR-invisible (divergence not detectable at LLVM IR level).

Name	Category	Verdict	CE	IR-Inv	ms
add_no_overflow	arith	diverg	a=2147483648, b=2147483648	yes	29
add_overflow_signed	overflow	diverg	a=2147483648, b=2147483648	yes	10
mul_overflow	overflow	diverg	a=4834196, b=124052570	yes	58
sub_overflow	overflow	diverg	a=-2147483648, b=1	yes	5
negate_min	overflow	diverg	a=-2147483648	yes	1
shift_overflow	overflow	diverg	shift=32	yes	3
max_function	ctrl	equiv	—	—	1
abs_function	ctrl	diverg	x=-2147483648	yes	2
bitwise_and	bitwise	equiv	—	—	1
bitwise_or	bitwise	equiv	—	—	1
unsigned_add	arith	equiv	—	—	1
int_to_uint_cast	cast	equiv	—	—	1
widening_cast	cast	equiv	—	—	1
truncating_cast	cast	equiv	—	—	1
div_by_zero	error	diverg	b=0	yes	1
int_min_div_neg1	overflow	diverg	a=-2147483648, b=-1	yes	1
float_add	float	diverg	FP precision	no	2000
float_mul_accumulate	float	diverg	FP precision	no	21
loop_bounded_ovf	loop	diverg	n=100000	yes	2
loop_sum_accumulate	loop	diverg	overflow	yes	1
char_check	string	equiv	—	—	1
array_access	memory	diverg	index OOB	no	1
safe_add_ovf_chk	ctrl	equiv	—	—	1
clamp_to_range	ctrl	equiv	—	—	1
popcount_naive	bitwise	equiv	—	—	1
rotate_left	bitwise	diverg	shift UB	yes	2
sign_extend_cast	cast	equiv	—	—	1
checked_mul_overflow	overflow	equiv	—	—	1
power_of_two_check	bitwise	equiv	—	—	1
byte_swap_32	bitwise	equiv	—	—	1
xor_equivalence	overflow	equiv	—	—	1
saturating_add	overflow	diverg	a=2147483647, b=1	yes	1

E CEGAR Experiment Details

E.1 Function List and Outcomes

The 20 functions used in the CEGAR experiment span control flow (2), loops (6), and overflow-sensitive arithmetic (7), plus 5 mixed-category functions. Table 8 shows per-function outcomes.

E.2 Failure Analysis

The 11 unrepaired functions share a common pattern: the commodity LLM (gpt-4.1-nano) struggles with overflow semantics that require *global* reasoning about value ranges. For example, `ipow` has two overflow sites (`result *= base` and `base *= base`) whose interaction means that fixing one with `wrapping_mul` changes the control flow for the other. The LLM tends to apply local fixes (e.g., replacing one operator) without understanding the cascading effects.

Table 8: CEGAR per-function outcomes. Iters = iterations to converge or reach limit.

Function	Category	Outcome	Iters
max	control flow	verified	1
clamp	control flow	verified	2
sum_n	loop	verified	3
factorial	loop	verified	2
count_bits	loop	verified	4
gcd	loop	verified	3
fib	loop	failed	5
collatz_steps	loop	failed	5
ipow	overflow	failed	5
mul_acc	overflow	failed	5
safe_add	overflow	failed	5
safe_sub	overflow	failed	5
safe_mul	overflow	failed	5
safe_neg	overflow	failed	5
safe_div	overflow	failed	5
abs_val	mixed	verified	2
sign	mixed	verified	1
midpoint	mixed	verified	3
popcount	mixed	failed	5
isqrt	mixed	failed	5

F Algorithms

F.1 SemanticConfig SMT Encoding

Algorithm 2 shows the SMT encoding procedure parameterized by the `SemanticConfig`.

F.2 Product Program Construction

Algorithm 3 shows the product program construction.

G Cross-Language Type Mapping

Table 9 shows the type mapping used by SemRec.

Table 9: C-to-Rust type mapping.

C Type	Rust Type
`int` / `signed int`	`i32`
`unsigned int`	`u32`
`long` / `long long`	`i64`
`unsigned long`	`u64`
`short` / `unsigned short`	`i16` / `u16`
`char` / `unsigned char`	`i8` / `u8`
`float` / `double`	`f32` / `f64`
`void`	`()`
`size_t` / `ssize_t`	`usize` / `isize`
`T*` (read-only)	`&T`
`T*` (mutated)	`&mut T`
`T*` (owned)	`Box<T>`

Algorithm 2 SemanticConfig-Parameterized SMT Encoding

procedure ENCODE($f_C, f_R, \sigma_C, \sigma_R$)
  $\vec{x} \leftarrow$ FreshBitVecs(params(f_C))
  $\phi_C \leftarrow$ ENCODEBODY($f_C, \vec{x}, \sigma_C$)
  $\phi_R \leftarrow$ ENCODEBODY($f_R, \vec{x}, \sigma_R$)
  wd $\leftarrow$ WELLDEFINED($\sigma_C, f_C, \vec{x}$)
  return wd $\wedge$ ($\phi_C \neq \phi_R$)
end procedure
procedure ENCODEOP($e_1, op, e_2, \sigma, \tau$)
  bv $\leftarrow$ BVOp(e_1, op, e_2)
  if $\sigma(op, \tau) =$ UB **then**
    return bv ▷ Add guard to WellDefined
  else if $\sigma(op, \tau) =$ WRAP **then**
    return bv ▷ BV arithmetic wraps by default
  else if $\sigma(op, \tau) =$ PANIC **then**
    return bv ▷ Set panic flag on overflow
  end if
end procedure

Algorithm 3 Product Program Construction

procedure BUILDPRODUCT(f_C, f_R)
  align $\leftarrow$ FUNCTIONALIGNER(f_C, f_R)
  $\vec{x} \leftarrow$ UNIFYINPUTS(align)
  $P_C \leftarrow$ LOWERTOIR($f_C, \vec{x}$, `c11()`)
  $P_R \leftarrow$ LOWERTOIR($f_R, \vec{x}$, `rust_release()`)
  coercions $\leftarrow$ COERCIONGENERATOR(P_C, P_R)
  $P \leftarrow$ JUXTAPOSE(P_C, P_R, coercions)
  P.assert(ret$_C \neq$ ret$_R$)
  return P
end procedure

H API Reference

SEMREC's core API consists of four modules:

```python
# Parsing
from src.frontend_c.parser import CParser
from src.frontend_rust.parser import RustParser

# IR lowering
from src.frontend_c.ir_lowering import CIRLowering
from src.frontend_rust.ir_lowering import RustIRLowering

# Product program construction
from src.product_program.product import ProductBuilder
from src.product_program.alignment import FunctionAligner
from src.product_program.coercion import CoercionGenerator

# SMT verification
from src.smt.solver import SMTSolver
from src.smt.encoder import SMTEncoder
from src.semantics.semantic_config import SemanticConfig
```

Basic usage.

```
# Parse
c_ast = CParser().parse("int f(int a) { return a+1; }")
r_ast = RustParser().parse("fn f(a: i32) -> i32 { a+1 }")

# Lower to IR
c_ir = CIRLowering(CTypeResolver()).lower(c_ast)
r_ir = RustIRLowering(RustTypeResolver()).lower(r_ast)

# Align and build product program
aligner = FunctionAligner()
alignment = aligner.align(c_ir, r_ir)
product = ProductBuilder().build(alignment)

# Verify with SemanticConfig
encoder = SMTEncoder(
    SemanticConfig.c11(), SemanticConfig.rust_release())
formula = encoder.encode(product)
result = SMTSolver().solve(formula)
# result.verdict: "equivalent" or "divergent"
# result.counterexample: Z3 model values
```

I Threats to Validity

Internal validity. The core benchmark uses hand-crafted pairs with known ground truth. This ensures correctness of verdicts but may not represent the difficulty distribution of real-world code. The CEGAR experiment uses LLM-generated translations to provide a more realistic evaluation.

External validity. All functions are single-function, $\leq$50 lines, operating on primitive types with bounded loop unrolling. Results may not generalize to functions with pointers, heap allocation, structs, or interprocedural calls.

Construct validity. All reported counterexamples are valid Z3 models, so each detected divergence corresponds to a concrete input that produces different outputs. However, some divergences may be unreachable in the context of a larger program (e.g., a caller that never passes `INT_MIN`).

CEGAR LLM sensitivity. The CEGAR results use a single commodity LLM (gpt-4.1-nano). Stronger models may achieve higher repair rates. The 45% success rate should be interpreted as a lower bound on what CEGAR-driven repair can achieve.